

使用雲端資料庫、無伺服器執行階段和開放原始碼整合功能，建立玩具店搜尋應用程式

來源：<https://codelabs.developers.google.com/toy-store-app?hl=zh-tw>

步驟數：16

在本程式碼研究室中，您將建構以 RAG 為基礎的向量搜尋應用程式，用於為客戶搜尋相符的玩具 (透過文字和圖片)、根據使用者要求建立自訂玩具，以及使用 AlloyDB、Gemini、Imagen、LangChain4j 和資料庫專用的 GenAI Toolbox 預測自訂玩具的價格。

目錄

1. [總覽](#)
2. [架構](#)
3. [事前準備](#)
4. [資料庫設定](#)
5. [資料擷取](#)
6. [為脈絡建立嵌入](#)
7. [執行向量搜尋](#)
8. [使用 LLM 驗證比對結果](#)
9. [將玩具搜尋應用程式無伺服器化](#)
10. [測試 Cloud Run 函式](#)
11. [建構網頁應用程式用戶端！](#)
12. [瞭解生成式 AI 詳細資料](#)
13. [資料庫專用的生成式 AI 工具箱](#)
14. [測試網頁應用程式](#)
15. [清除所用資源](#)
16. [恭喜](#)

1. 總覽

想像一下，你走進虛擬或實體玩具店，輕鬆找到最完美的禮物。你可以描述想找的玩具、上傳玩具圖片，甚至設計自己的玩具，商店會立即瞭解你的需求，提供量身打造的體驗。這不是未來的幻想，而是由 AI、雲端技術和個人化電子商務願景所實現的現實。

挑戰：光是找到符合想像的完美產品，就可能是一大難題。一般搜尋字詞、關鍵字和模糊搜尋通常不夠精確，瀏覽無止盡的頁面可能很無聊，而且想像中的商品與實際可用的商品之間落差太大，可能會令人感到沮喪。

解決方案：這個示範應用程式直接解決這項挑戰，運用 AI 的強大功能，根據搜尋情境提供相符的產品，並自訂生成結果，打造真正個人化且流暢的體驗。

建構項目

本實驗室的學習內容包括：

1. 建立 AlloyDB 執行個體並載入玩具資料集
2. 在 AlloyDB 中啟用 pgvector 和生成式 AI 模型擴充功能
3. 從產品說明生成嵌入，並對使用者搜尋文字執行即時餘弦相似度搜尋
4. 呼叫 Gemini 2.0 Flash，描述使用者上傳的圖片，以便搜尋相關玩具
5. 根據使用者興趣，呼叫 Imagen 3 自訂建立玩具
6. 叫用使用 Gen AI Toolbox for Databases 建立的價格預測工具，取得自訂玩具的價格詳細資料
7. 在無伺服器 Cloud Run 函式中部署解決方案

需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
 - 已啟用計費功能的 Google Cloud 專案。
-

2. 架構

資料流程：讓我們進一步瞭解資料在系統中的流動方式：

1. 透過 AI 輔助 RAG (檢索增強生成) 進行情境搜尋

舉例來說，系統不會只搜尋「紅色汽車」，而是會瞭解以下資訊：

「適合 3 歲男孩的小車」。

以 **AlloyDB** 為基礎：我們使用 Google Cloud 的全代管 PostgreSQL 相容資料庫 AlloyDB，儲存玩具資料，包括說明、圖片網址和其他相關屬性。

pgvector 適用於語意搜尋：pgvector 是 PostgreSQL 擴充功能，可儲存玩具說明和使用者搜尋查詢的向量嵌入。這樣一來，系統就能執行語意搜尋，也就是瞭解字詞背後的意義，而不只是完全相符的關鍵字。

餘弦相似度 (適用於關聯性)：我們會使用餘弦相似度，測量使用者搜尋向量與玩具說明向量之間的語意相似度，找出最相關的結果。

ScaNN 索引兼顧速度與準確率：為確保結果快速準確，我們整合了 ScaNN (可擴充的近鄰) 索引，尤其是在玩具目錄不斷擴充的情況下，這項功能大幅提升了向量搜尋的效率和召回率。

2. 使用 Gemini 2.0 Flash 進行圖片搜尋及理解

假設使用者想上傳一張熟悉玩具的圖片，並用這張圖片進行搜尋，而不是輸入文字。使用者可以上傳喜歡的玩具圖片，並取得相關功能。我們運用 LangChain4j 叫用的 Google Gemini 2.0 Flash 模型分析圖片，並擷取相關脈絡，例如玩具的顏色、材質、類型和適用年齡層。

3. Building Your Dream Toy Customized with Generative AI: Imagen 3

使用者決定自行製作玩具時，才是真正發揮創意的時候。我們讓他們使用 Imagen 3，透過簡單的文字提示詞描述夢想中的玩具。想像一下，只要說出「我想要一隻有紫色翅膀和友善臉龐的絨毛龍」，螢幕上就會出現這隻龍！Imagen 3 接著會生成自訂設計玩具的圖片，讓使用者清楚看到自己的創作。

4. 運用 Agent 和 MCP Toolbox for Databases 進行價格預測

我們導入了價格預測功能，可估算生產客製化玩具的成本。這項功能由內建複雜價格計算工具的智慧代理程式提供。

MCP Toolbox for Databases：這個代理使用 Google 全新開放原始碼工具 MCP Toolbox for Databases，與資料庫完美整合。這樣一來，代理程式就能存取材料成本、製造程序和其他相關因素的即時資料，提供準確的價格估算。詳情請參閱 [這篇文章](#)。

5. Java Spring Boot、Gemini Code Assist 和 Cloud Run，可簡化開發作業並部署無伺服器應用程式

整個應用程式都是使用 Java Spring Boot 建構，這個架構功能強大且可擴充。我們在整個開發過程中運用 Gemini Code Assist，特別是前端開發，大幅加快開發週期並提升程式碼品質。我們使用 Cloud Run 部署整個應用程式，並使用 Cloud Run Functions 將資料庫和代理程式功能部署為獨立端點。

3. 事前準備

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境，預先載入了 bq。按一下 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 在 Cloud Shell 終端機中逐一執行下列指令，啟用必要的 API：

您也可以執行單一指令來執行下列操作，但如果您是試用帳戶使用者，嘗試大量啟用這些功能時可能會遇到配額問題。因此，每個指令都會單獨列於一行。

```
gcloud services enable alloydb.googleapis.com
gcloud services enable compute.googleapis.com
gcloud services enable cloudresourcemanager.googleapis.com
gcloud services enable servicenetworking.googleapis.com
gcloud services enable run.googleapis.com
gcloud services enable cloudbuild.googleapis.com
gcloud services enable cloudfunctions.googleapis.com
gcloud services enable aiplatform.googleapis.com
```

除了使用 gcloud 指令，您也可以透過控制台搜尋各項產品，或使用這個[連結](#)。

如果遺漏任何 API，您隨時可以在導入過程中啟用。

如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

步驟 4 · 約 0 分鐘

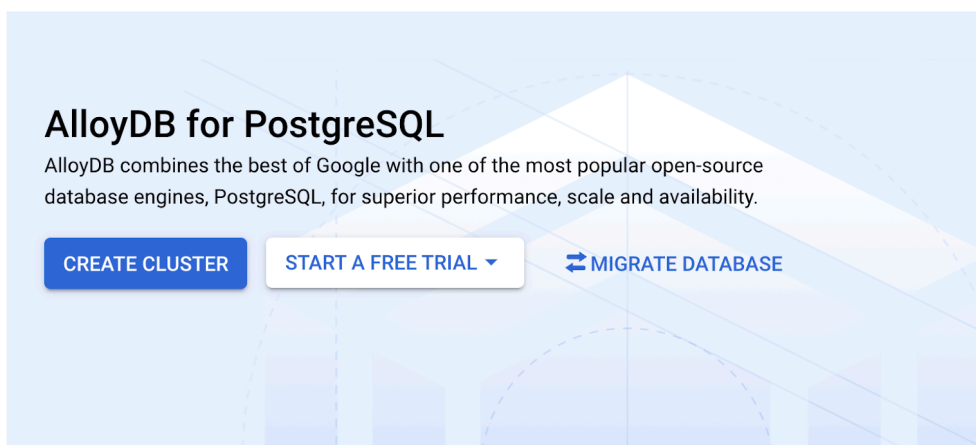
4. 資料庫設定

在本實驗室中，我們將使用 AlloyDB 做為資料庫，存放玩具店資料。並使用「叢集」保存所有資源，例如資料庫和記錄檔。每個叢集都有一個「主要執行個體」，可做為資料的存取點。資料表會保存實際資料。

我們來建立 AlloyDB 叢集、執行個體和資料表，以便載入電子商務資料集。

建立叢集和執行個體

1. 在 Cloud 控制台中前往 AlloyDB 頁面。如要在 Cloud 控制台尋找大部分的頁面，只要使用控制台的搜尋列搜尋即可。
2. 選取該頁面中的「建立叢集」：



3. 您會看到如下所示的畫面。使用下列值建立叢集和執行個體 (如果您要從存放區複製應用程式程式碼，請確保值相符)：

- 叢集 ID：「vector-cluster」
- password: "alloydb"
- 與 PostgreSQL 15 相容
- 區域：「us-central1」
- 網路：「default」

Configure your cluster

Cluster ID *

Use lowercase letters, numbers, and hyphens. Start with a letter.

Password *



[GENERATE](#)

Set a password for the default "postgres" user. A password is required for the user to log in.

Database version *

PostgreSQL 15 compatible



Location

For better performance, keep your data close to the services that need it. Choice is permanent.

Region *

us-east4 (Northern Virginia)



Networking

Clusters can only be configured with a private IP network path. [Learn more](#)

Network *



[SHOW ADVANCED OPTIONS](#)

4. 選取預設網路後，你會看到如下畫面。

選取「設定連線」。

Networking

Clusters can only be configured with a private IP network path. [Learn more](#)

Network *

default



Private services access connection required

Your network "default" requires a private services access connection. This connection enables your services to communicate exclusively by using internal IP addresses. [Learn more](#)

[SET UP CONNECTION](#)

5. 然後選取「使用系統自動分配的 IP 範圍」並繼續。確認資訊後，選取「建立連結」。

✓ **Enable Service Networking API**

2 **Allocate an IP range**

Google will use this allocated IP range to create subnets.

☐ Select one or more existing IP ranges or create a new one

Select or create an IP range ▼

☒ Use an automatically allocated IP range

Google will automatically allocate an IP range of prefix-length /20 and use the name "default-ip-range".

CONTINUE

3 **Create a connection**

CREATE CONNECTION

CANCEL

6. 設定網路後，即可繼續建立叢集。按一下「CREATE CLUSTER」(建立叢集)，完成叢集設定，如下所示：

Configure your primary instance

Name your instance

Give your instance an ID.

Instance ID *
shopping-cluster-primary

Choice is permanent. Use lowercase letters, numbers, and hyphens. Start with a letter.

Zonal availability

- ☐ Single zone
In case of outage, no failover. Not recommended for production.
- ☒ Multiple zones (Highly available)
Automatic failover to another zone within your selected region. Recommended for production instances. Increases cost.

Machine

AlloyDB's superior performance lets you run your existing workloads on smaller machines.

Machine Type *
8 vCPU, 64 GB

Connectivity

Private IP Connectivity

- ☒ Enable Private IP (always enabled)
Connect from applications and clients within the associated network. [Learn more](#)

CREATE CLUSTER

CANCEL

請務必將執行個體 ID 變更為

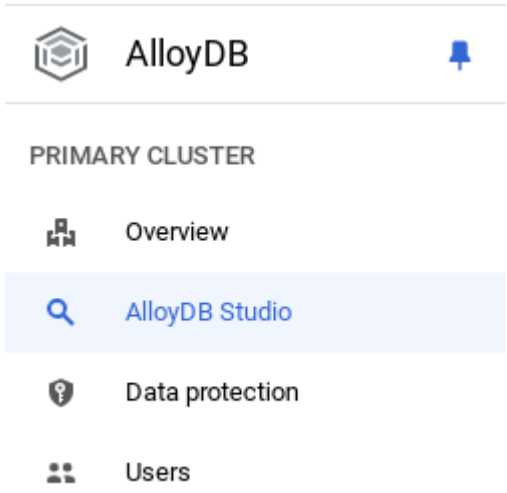
vector-instance

如果無法變更，請記得在所有後續參照中變更執行個體 ID。

請注意，建立叢集約需 10 分鐘。成功後，畫面上會顯示您剛建立的叢集總覽。

5. 資料擷取

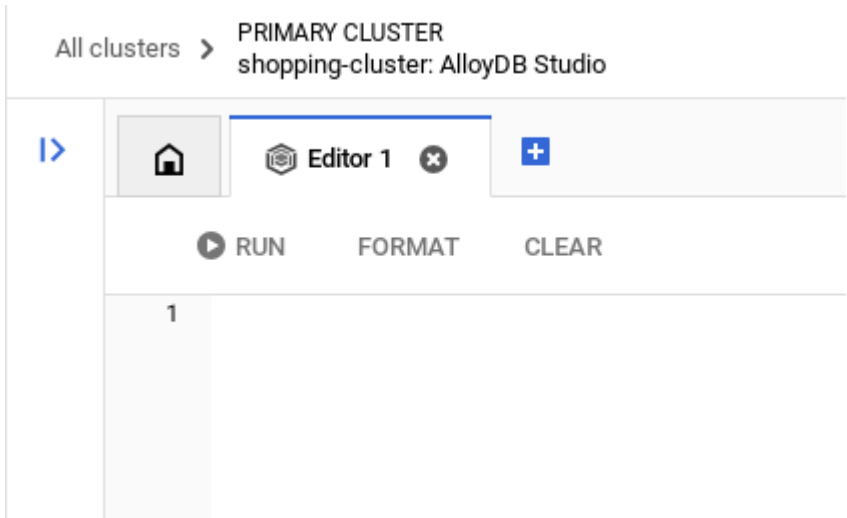
現在要新增包含商店資料的表格。前往 AlloyDB，選取主要叢集，然後選取 AlloyDB Studio：



您可能需要等待執行個體建立完成。完成後，請使用建立叢集時建立的憑證登入 AlloyDB。使用下列資料向 PostgreSQL 進行驗證：

- 使用者名稱：「postgres」
- 資料庫：「postgres」
- 密碼：「alloydb」

成功驗證 AlloyDB Studio 後，即可在編輯器中輸入 SQL 指令。如要新增多個編輯器視窗，請按一下最後一個視窗右側的加號。



您會在編輯器視窗中輸入 AlloyDB 指令，並視需要使用「執行」、「格式化」和「清除」選項。

啟用擴充功能

我們會使用 `pgvector` 和 `google_ml_integration` 擴充功能建構這個應用程式。[pgvector 擴充功能](#)可讓您儲存及搜尋向量嵌入。[google_ml_integration](#) 擴充功能提供多種函式，可存取 Vertex AI 預測端點，在 SQL 中取得預測結果。執行下列 DDL，[啟用](#)這些擴充功能：

```
CREATE EXTENSION IF NOT EXISTS google_ml_integration CASCADE;  
CREATE EXTENSION IF NOT EXISTS vector;
```

如要查看資料庫已啟用的擴充功能，請執行下列 SQL 指令：

```
select extname, extversion from pg_extension;
```

建立資料表

使用下列 DDL 陳述式建立資料表：

```
CREATE TABLE toys ( id VARCHAR(25), name VARCHAR(25), description VARCHAR(20000), quantity  
INT, price FLOAT, image_url VARCHAR(200), text_embeddings vector(768)) ;
```

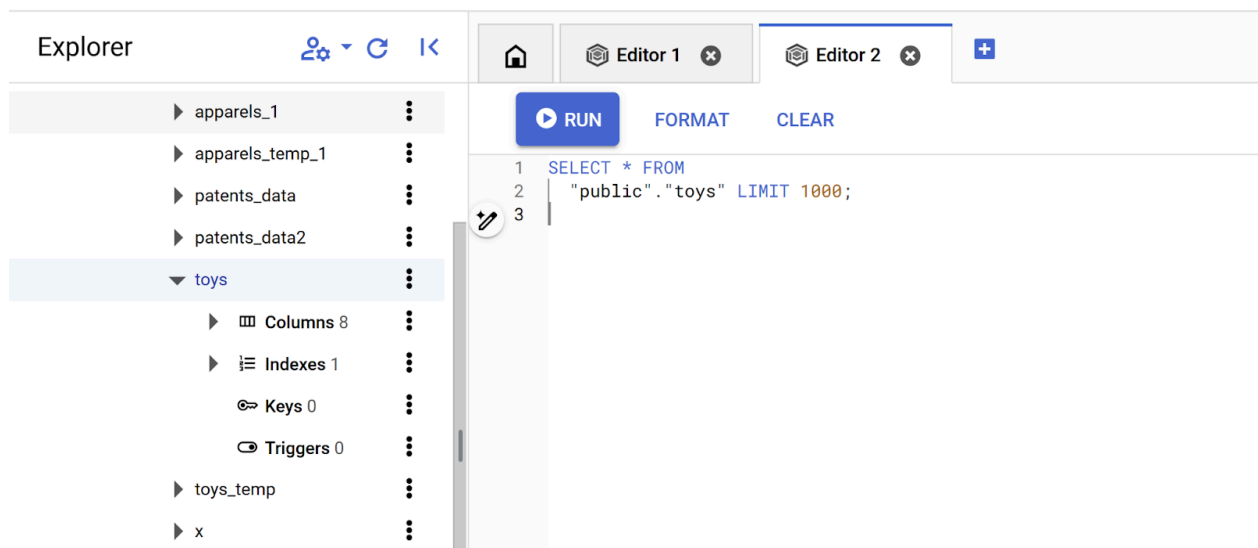
成功執行上述指令後，您應該就能在資料庫中查看資料表。

擷取資料

在本實驗室中，我們已在這個 [SQL 檔案](#) 中提供約 72 筆記錄的測試資料。其中包含 `id`, `name`, `description`, `quantity`, `price`, `image_url` 欄位。其他欄位會在實驗室的後續部分填寫。

從中複製前 5 行/插入陳述式，然後將這些行貼到空白的編輯器分頁中，並選取「執行」。如果不是使用試用帳單帳戶，您可能可以複製所有插入陳述式並執行。

如要查看資料表內容，請展開「Explorer」專區，直到看到名為 `apparels` 的資料表為止。選取三點圖示 (:)，即可查看查詢資料表的選項。系統會在新編輯器分頁中開啟 `SELECT` 陳述式。



授予權限

執行下列陳述式，將 `embedding` 函式的執行權授予使用者 `postgres`：

```
GRANT EXECUTE ON FUNCTION embedding TO postgres;
```

為 AlloyDB 服務帳戶授予 Vertex AI 使用者角色

前往 Cloud Shell 終端機並輸入下列指令：

```
PROJECT_ID=$(gcloud config get-value project)

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:service-${gcloud projects describe $PROJECT_ID --
format="value(projectNumber)"}@gcp-sa-alloydb.iam.gserviceaccount.com" \
  --role="roles/aiplatform.user"
```

6. 為脈絡建立嵌入

電腦處理數字比處理文字容易得多。嵌入系統會將文字轉換為一系列浮點數，這些數字應代表文字，無論文字的措辭、使用的語言等為何。

例如描述海邊的地點。例如「水上」、「海濱」、「從房間走到海邊」、「sur la mer」、「на берегу океана」等。這些字詞看起來不同，但元件資訊或機器學習術語 (即嵌入) 應該非常接近。

資料和內容都準備就緒後，我們將執行 SQL，將產品說明的嵌入內容新增至 `embedding` 欄位中的資料表。您可以使用多種嵌入模型。我們使用 Vertex AI 的 `text-embedding-005`。請務必在整個專案中使用相同的嵌入模型！

注意：如果您使用先前建立的 Google Cloud 專案，可能需要繼續使用舊版文字嵌入模型，例如 `textembedding-gecko`。

返回 AlloyDB Studio 分頁，然後輸入下列 DML：

```
UPDATE toys set text_embeddings = embedding( 'text-embedding-005', description);
```

再次查看 `toys` 表格，即可看到一些嵌入內容。請務必重新執行 `SELECT` 陳述式，查看變更。

```
SELECT id, name, description, price, quantity, image_url, text_embeddings FROM toys;
```

這應該會傳回玩具說明的嵌入向量，看起來像是浮點數陣列，如下所示：

All clusters > PRIMARY CLUSTER
vector-cluster: AlloyDB Studio

Editor 1 Editor 2

RUN FORMAT CLEAR

1 SELECT id, name, description, price, quantity, image_url, text_embeddings FROM toys;

RESULTS

id	name	description	price	quantity	image_url	text_embeddings	
1	car_1	Orange vi...	49.99	25	https://storage.googleapis.com/img_public_test/toys/car_1.png	[0.006680974,0.017135253,-0.0020700733,0.000368	✓
2	car_2	Red vinta...	49.99	20	https://storage.googleapis.com/img_public_test/toys/car_2.png	[0.002093151,0.035488974,0.007591748,0.02496356	✓
3	car_3	Orange o...	59.99	22	https://storage.googleapis.com/img_public_test/toys/car_3.png	[0.023304963,0.01982177,0.007610128,0.005366554	✓
4	car_4	Silver vin...	48.99	28	https://storage.googleapis.com/img_public_test/toys/car_4.png	[0.0055077737,0.01959007,0.0049791783,-0.014084	✓
5	car_5	Red toy c...	69.99	22	https://storage.googleapis.com/img_public_test/toys/car_5.png	[0.009379575,0.05221759,0.017651267,-0.02488539	✓

注意：在免費方案下建立的 Google Cloud 專案，可能會遇到配額問題，也就是每秒允許的嵌入要求數量。建議您使用 ID 的篩選查詢，然後在產生嵌入內容時，選擇性地選取 1 到 5 筆記錄等。

7. 執行向量搜尋

資料表、資料和嵌入都已準備就緒，現在就來對使用者搜尋文字執行即時向量搜尋。

假設使用者提出以下問題：

「I want a white plush teddy bear toy with a floral pattern」。

您可以執行下列查詢來找出相符項目：

```
select * from toys
ORDER BY text_embeddings <=> CAST(embedding('text-embedding-005', 'I want a white plush teddy
bear toy with a floral pattern') as vector(768))
LIMIT 2;
```

讓我們詳細瞭解這項查詢：

在這項查詢中，

- 1. 使用者的搜尋文字為：「I want a white plush teddy bear toy with a floral pattern.」
- 2. 我們要在 embedding() 方法中使用模型 text-embedding-005，將其轉換為嵌入。上一個步驟中，我們已將嵌入函式套用至表格中的所有項目，因此這個步驟應該很熟悉。
- 3. 「<=>」代表使用「餘弦相似度」距離方法。如要查看所有可用的相似度測量方式，請參閱 [pgvector 說明文件](#)。
- 4. 我們會將嵌入方法的結果轉換為向量型別，使其與資料庫中儲存的向量相容。
- 5. LIMIT 5 代表我們要為搜尋文字擷取 5 個最鄰近的項目。

結果如下所示：

Editor 2

RUN SELECTED

FORMAT

CLEAR

Syntax error at or near "("

```
1 select * from toys
2 ORDER BY text_embeddings <=> CAST(embedding('text-embedding-005', 'white plush teddy bear toy with floral pattern') as vector(768))
3 LIMIT 5;
```

RESULTS

id	name	description	quantity	price	image_url	image_embeddings	text_embeddings	
12	ima...	White pl...	85	15.99	https://storage.googleapis.com/img_public_test/toys/ima...		[0.013211509,-0.004979305,0.02210...	✓
56	ima...	Large w...	100	59.99	https://storage.googleapis.com/img_public_test/toys/ima...		[0.05454888,-0.004123779,0.01804...	✓
21	ima...	Yellow p...	75	15.99	https://storage.googleapis.com/img_public_test/toys/ima...		[0.021026356,0.017582001,0.00400...	✓
20	ima...	Yellow p...	70	15.99	https://storage.googleapis.com/img_public_test/toys/ima...		[0.021026356,0.017582001,0.00400...	✓
19	ima...	Yellow p...	65	15.99	https://storage.googleapis.com/img_public_test/toys/ima...		[0.021003863,0.034315277,0.00511...	✓

如您在結果中看到的，相符項目與搜尋文字相當接近。請嘗試變更文字，看看結果會如何變化。

重要注意事項：

假設我們現在想使用 **ScaNN** 索引，提高這項向量搜尋結果的效能 (查詢時間)、效率和召回率。請參閱這篇 [網誌](#) 中的步驟，比較有索引和沒有索引的結果差異。

選用步驟：使用 ScaNN 索引提升效率和召回率

如果記錄數少於 100，請略過這個步驟。

為方便您操作，以下列出建立索引的步驟：

- 1. 由於我們已建立叢集、例項、內容和嵌入，因此只需使用下列陳述式安裝 ScaNN 擴充功能：

```
CREATE EXTENSION IF NOT EXISTS alloydb_scann;
```

2. 接著，我們要建立索引 (ScaNN)：

```
CREATE INDEX toysearch_index ON toys  
USING scann (text_embeddings cosine)  
WITH (num_leaves=9);
```

在上述 DDL 中，`toysearch_index` 是索引的名稱

「toys」是我的表格

「scann」是索引方法

「embedding」是我要建立索引的表格欄

「cosine」是我要用於索引的距離方法

「8」是要套用至這個索引的分區數量。請設為 1 到 1048576 之間的任何值。如要進一步瞭解如何決定這個值，請參閱「[調整 ScaNN 索引](#)」。

我使用資料點數的平方根，如 [ScaNN 存放區](#) 中建議 (分區時，`num_leaves` 應約為資料點數的平方根)。

3. 使用下列查詢檢查索引是否已建立：

```
SELECT * FROM pg_stat_ann_indexes;
```

4. 使用與不含索引時相同的查詢，執行向量搜尋：

```
select * from toys  
ORDER BY text_embeddings <=> CAST(embedding('text-embedding-005', 'I want a white plush teddy  
bear toy with a floral pattern') as vector(768))  
LIMIT 5;
```

上述查詢與實驗室步驟 8 中使用的查詢相同。不過，現在我們已為該欄位建立索引。

5. 使用簡單的搜尋查詢測試索引，並捨棄索引：

這個用途只有 72 筆記錄，因此索引不會真正生效。在[其他用途](#)中進行的測試結果如下：

對已建立索引的嵌入項目資料執行相同的 Vector Search 查詢，可獲得高品質的搜尋結果，並提升效率。使用索引後，效率大幅提升 (執行時間：**不使用 ScaNN** 時為 10.37 毫秒，**使用 ScaNN** 時為 0.87 毫秒)。如要進一步瞭解這個主題，請參閱這篇[網誌](#)。

8. 使用 LLM 驗證比對結果

在繼續操作並建立服務，將最佳結果傳回應用程式之前，請先使用生成式 AI 模型驗證這些潛在回覆是否確實相關，且可安全地與使用者分享。

確認執行個體已設定為 Gemini

首先，請檢查叢集和執行個體是否已啟用 Google ML 整合。在 AlloyDB Studio 中，輸入下列指令：

```
show google_ml_integration.enable_model_support;
```

如果值顯示為「on」，您可以略過接下來的 2 個步驟，直接設定 AlloyDB 和 Vertex AI 模型整合。

- 1. 前往 AlloyDB 叢集的主要執行個體，然後按一下「編輯主要執行個體」

Clusters

[+ CREATE CLUSTER](#) [MIGRATE DATA](#) [INSTALL ALLOYDB OMNI](#) [PURCHASE ALLOYDB OMNI](#) [EXPLORE GEMINI](#)

Resource name	Resource type	Location	Read pool nodes	Maintenance	Issues	Actions
▼ gccd-kol	Primary cluster	us-east4				⋮
gccd-kol	Primary instance	us-east4-a			Allows direct unencrypted connect Auditing not enabled No password policy	⋮
▼ onlineboutique-cluster	Primary cluster	us-central1				⋮
onlineboutique-instance	Primary instance	us-central1-a			Auditing not enabled No password policy	⋮
onlineboutique-instance-replica	Read pool instance	us-central1	2		Auditing not enabled No password policy	⋮
patent-gedd	Primary cluster	us-central1				⋮
shopping-cluster	Primary cluster	us-central1				⋮
▼ vector-cluster	Primary cluster	us-central1				⋮
vector-instance	Primary instance	us-central1-a			Allows direct unencrypted connect Auditing not enabled No password policy	⋮

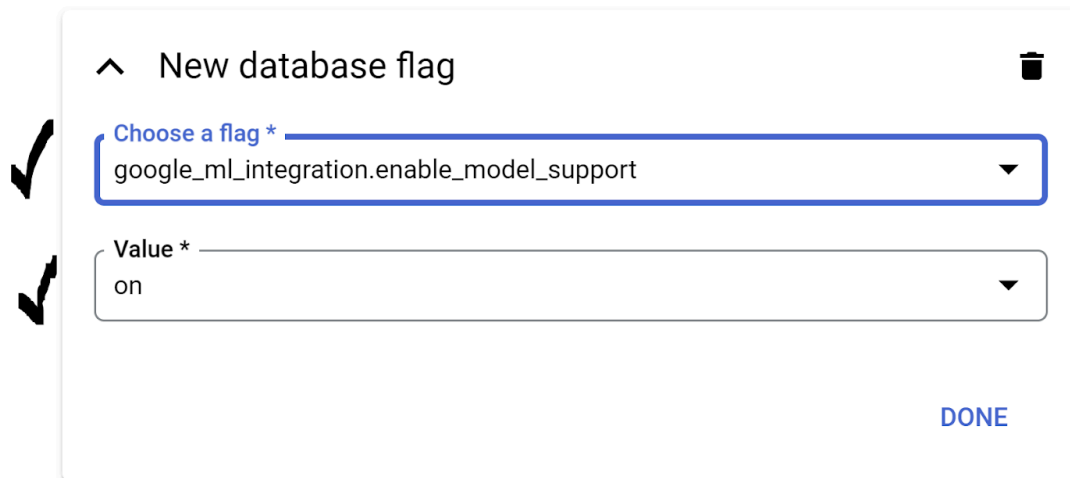
- 2. 前往「進階設定選項」中的「Flags」部分。並確認 `google_ml_integration.enable_model_support flag` 設為「on」，如下所示：

← Edit primary instance

Flags

Use flags to customise your instance, if needed. [Learn more](#)

Enable the columnar engine flag to improve performance and query speed of HTAP and OLAP workloads. [Learn more](#)



^ New database flag

Choose a flag *
google_ml_integration.enable_model_support

Value *
on

DONE

如果未設為「on」，請設為「on」，然後按一下「UPDATE INSTANCE」按鈕。這個步驟需要幾分鐘才能完成。

整合 AlloyDB 和 Vertex AI 模型

現在您可以連線至 AlloyDB Studio，並執行下列 DML 陳述式，從 AlloyDB 設定 Gemini 模型存取權 (請使用專案 ID，如圖所示)。執行指令前，系統可能會警告您有語法錯誤，但指令應該可以正常執行。

首先，我們建立 Gemini 1.5 模型連線，如下所示。請記得將下方指令中的 `$PROJECT_ID` 替換為您的 Google Cloud 專案 ID。

```
CALL
google_ml.create_model( model_id => 'gemini-1.5',
  model_request_url => 'https://us-central1-
aiplatform.googleapis.com/v1/projects/$PROJECT_ID/locations/us-
central1/publishers/google/models/gemini-1.5-pro:streamGenerateContent',
  model_provider => 'google',
  model_auth_type => 'alloydb_service_agent_iam');
```

您可以在 AlloyDB Studio 中執行下列指令，檢查設定可存取的模型：

```
select model_id,model_type from google_ml.model_info_view;
```

最後，我們需要授予資料庫使用者執行 `ml_predict_row` 函式的權限，透過 Google Vertex AI 模型執行預測。執行下列指令：

```
GRANT EXECUTE ON FUNCTION ml_predict_row to postgres;
```

注意：如果您使用現有的 Google Cloud 雲端專案和現有的 AlloyDB 叢集/執行個體，且這些資源是先前建立的，可能需要捨棄 gemini-1.5 模型舊的參照，並使用上述 CALL 陳述式重新建立，然後再次對函式 ml_predict_row 執行授權，以免在後續叫用 gemini-1.5 時發生問題。

評估回覆

雖然我們會在下一節中使用一個大型查詢，確保查詢的回應合理，但查詢可能難以理解。我們現在就來看看這些片段，並在幾分鐘內瞭解它們如何整合。

1. 首先，我們會向資料庫傳送要求，取得與使用者查詢最接近的 10 個結果。
2. 為判斷回覆的有效性，我們會使用外部查詢，說明如何評估回覆。查詢時會使用搜尋文字的 recommended_text 欄位，以及內部資料表的 content (玩具說明欄位)。
3. 我們會根據這項指標，審查傳回的回覆是否「良好」。
4. predict_row 會以 JSON 格式傳回結果。程式碼「-> 'candidates' -> 0 -> 'content' -> 'parts' -> 0 -> 'text'」用於從該 JSON 擷取實際文字。如要查看實際傳回的 JSON，可以移除這段程式碼。
5. 最後，如要取得 LLM 回覆，請使用 REGEXP_REPLACE(gemini_validation, '[^a-zA-Z,:]', '', 'g') 擷取回覆。

```

SELECT id,
       name,
       content,
       quantity,
       price,
       image_url,
       recommended_text,
       REGEXP_REPLACE(gemini_validation, '[^a-zA-Z,: ]', '', 'g') AS gemini_validation
FROM (SELECT id,
            name,
            content,
            quantity,
            price,
            image_url,
            recommended_text,
            CAST(ARRAY_AGG(LLM_RESPONSE) AS TEXT) AS gemini_validation
      FROM (SELECT id,
                  name,
                  content,
                  quantity,
                  price,
                  image_url,
                  recommended_text,
                  json_array_elements(google_ml.predict_row(model_id => 'gemini-1.5',
                                                             request_body => CONCAT('{
"contents": [ { "role": "user", "parts": [ { "text": "User wants to buy a toy and this is the
description of the toy they wish to buy: ',
                                                             recommended_text,
                                                             '. Check if the following
product items from the inventory are close enough to really, contextually match the user
description. Here are the items: ',
                                                             content,
                                                             '. Return a ONE-LINE response with 3 values: 1) MATCH: if
the 2 contexts are reasonably matching in terms of any of the color or color family specified
in the list, approximate style match with any of the styles mentioned in the user search
text: This should be a simple YES or NO. Choose NO only if it is completely irrelevant to
users search criteria. 2) PERCENTAGE: percentage of match, make sure that this percentage is
accurate 3) DIFFERENCE: A clear one-line easy description of the difference between the 2
products. Remember if the user search text says that some attribute should not be there, and
the record has it, it should be a NO match. " } ] } ] }'))::JSON)) -> 'candidates' -> 0 ->
'content' -> 'parts' -> 0 -> 'text' :: TEXT AS LLM_RESPONSE
                FROM (SELECT id,
                            name,
                            description AS content,
                            quantity,
                            price,
                            image_url,
                            'Pink panther standing' AS recommended_text
                      FROM toys
                      ORDER BY text_embeddings <=> embedding('text-embedding-005',
                                                             'Pink panther
standing'))::VECTOR
            )
          )
    )

```

```

LIMIT 1) AS xyz) AS X
GROUP BY id,
        name,
        content,
        quantity,
        price,
        image_url,
        recommended_text) AS final_matches

-- WHERE REGEXP_REPLACE(gemini_validation, '^[a-zA-Z,: ]', '', 'g') LIKE '%MATCH%:YES%';

```

雖然這看起來還是很複雜，但希望您能從中瞭解更多資訊。結果會顯示是否有相符的內容、相符程度百分比，以及評分說明。

請注意，Gemini 模型預設會啟用串流功能，因此實際回應會分散在多行中：

Home
 Editor 2
 Editor 1

▶ RUN
FORMAT
CLEAR
✔ Va

```
inventory are close enough to really, contextually match the user description. Here are the items: ',
content,
'. Return a ONE-LINE response with 3 values: 1)
MATCH: if the 2 contexts are reasonably matching in terms of any of the color or color family specified in the list, approximate style match with
any of the styles mentioned in the user search text: This should be a simple YES or NO. Choose NO only if it is completely irrelevant to users
search criteria. 2) PERCENTAGE: percentage of match, make sure that this percentage is accurate 3) DIFFERENCE: A clear one-line easy description of
the difference between the 2 products. Remember if the user search text says that some attribute should not be there, and the record has it, it
should be a NO_match. " } | } | } }::JSON)) -> 'candidates' -> 0 -> 'content' -> 'parts' -> 0 -> 'text' :: TEXT AS LLM_RESPONSE
```

RESULTS

60	image (59)	Pink toy of the Pink Panther with yellow eyes and black whiskers.	60	49.99	https://storage.googleapis.com/img_public_test/toys/image (59).png	Pink panther standing	product description doesn't specify if the toy is poseable in that way n	^
62	imag...	Pink toy of the...	60	49.99	https://storage.googleapis.com/img_public_test/toys/imag...	Pink panther sta...	MATCH.: YES PERCE...	v
59	imag...	Pink toy of the...	100	49.99	https://storage.googleapis.com/img_public_test/toys/imag...	Pink panther sta...	MATCH.: YES, PERCE...	v

9. 將玩具搜尋應用程式無伺服器化

準備好將這個應用程式帶到網路上嗎？請按照下列步驟，使用 Cloud Run 函式建立無伺服器 Knowledge Engine：

1. 前往 Google Cloud 控制台的 Cloud Run Functions，**建立新的 Cloud Run Function**，或使用連結：
<https://console.cloud.google.com/functions/add>。
2. 選取「Cloud Run function」做為環境。提供函式名稱「**get-toys-alloydb**」，並選擇「us-central1」做為區域。將「驗證」設為「允許未經驗證的叫用」，然後按一下「下一步」。選擇「Java 17」做為執行階段，並選擇「Inline Editor」做為原始碼。
3. 根據預設，進入點會設為「`gcfv2.HelloHttpFunction`」。請將 Cloud Run 函式 `HelloHttpFunction.java` 和 `pom.xml` 中的預留位置程式碼，分別替換為 [HelloHttpFunction.java](#) 和 [pom.xml](#) 中的程式碼。
4. 請記得在 Java 檔案中，將 `<<YOUR_PROJECT>>` 預留位置和 AlloyDB 連線憑證換成您的值。AlloyDB 憑證是我們在本程式碼研究室一開始使用的憑證。如果使用不同的值，請在 Java 檔案中修改。
5. 按一下 [Deploy] (部署)。

部署完成後，為了允許 Cloud 函式存取 AlloyDB 資料庫執行個體，我們將建立 **VPC 連接器**。

重要步驟：

開始部署後，您應該就能在 Google [Cloud Run Functions 控制台](#)中看到函式。搜尋新建立的函式 (`get-toys-alloydb`)，點選該函式，然後按一下「編輯」並變更下列項目：

1. 前往「執行階段、建構作業、連線和安全性設定」
2. 將逾時時間延長至 180 秒
3. 前往「連線」分頁：



Runtime, build, connections and security settings



BUILD

CONNECTIONS

SECURITY AND IMAGE REPO



Ingress settings ?

- ☒ Allow all traffic
- ☐ Allow internal traffic only
Only traffic from VPC networks in the same project or the same VPC SC perimeter is allowed.
- ☐ Allow internal traffic and traffic from Cloud Load Balancing
Traffic from VPC networks in the same project, the same VPC SC perimeter or from Cloud Load Balancing is allowed.

Egress settings ?

By default, your function can send requests to the Internet, but not to resources in VPC networks. To send requests to resources in your VPC network, create or select a VPC connector already created in the same region as the function.

Network

default: Serverless VPC Access connector alloydb-test-conn (in this proje... ▼

Access resources on a VPC. [Learn more](#)

- ☐ Only route requests to private IPs through the VPC connector
- ☒ Route all traffic through the VPC connector

- 在 Ingress 設定下方，確認已選取「允許所有流量」。
- 在「輸出設定」下方，按一下「網路」下拉式選單，然後選取「新增虛擬私有雲連接器」選項，並按照彈出式對話方塊中顯示的操作說明進行操作：

Create connector

Name *

Region *

us-central1

A region is a specific geographical location where you can run your resources.

Network *

default

Subnet *

Select an unused /28 subnet or create a new one by entering an unused /28 IP range. The VPC Connector will create connector instances on this subnet.

▼ SHOW SCALING SETTINGS

CREATE

CANCEL

6. 提供 VPC 連接器的名稱，並確認區域與執行個體相同。將「網路」值保留為預設值，並將「子網路」設為「自訂 IP 範圍」，IP 範圍為 10.8.0.0 或類似的可用範圍。
7. 展開「顯示縮放設定」，確認設定完全符合下列條件：

Scaling Settings

Minimum instances *

2

The minimum number of instances provisioned at any time. The connector will automatically scale out if more capacity is needed. Minimum number of instances cannot be decreased later. Larger values increase your cost.

Maximum instances *

3

The maximum number of instances provisioned at any time. The connector will not automatically scale out above this value. Maximum number of instances cannot be decreased later. This setting limits your maximum cost.



Connectors don't scale down automatically or manually. Once the connector has reached maximum number of instances it will remain at this number.

Instance type *

f1-micro

Larger instances support higher bandwidth but raise your costs. [Learn more](#)

^ HIDE SCALING SETTINGS

CREATE

CANCEL

8. 按一下「建立」，這個連接器現在應該會列在輸出設定中。
 9. 選取新建立的連接器
 10. 選擇透過這個虛擬私有雲連接器轉送所有流量。
 11. 依序點選「NEXT」和「DEPLOY」。
-

步驟 10 · 約 0 分鐘

10. 測試 Cloud Run 函式

部署更新後的 Cloud 函式後，您應該會看到產生的端點。複製該值，並替換下列指令中的值：

或者，您也可以按照下列步驟測試 Cloud Run 函式：

```
PROJECT_ID=$(gcloud config get-value project)

curl -X POST <<YOUR_ENDPOINT>> \
  -H 'Content-Type: application/json' \
  -d '{"search":"I want a standing pink panther toy"}' \
  | jq .
```

結果如下：

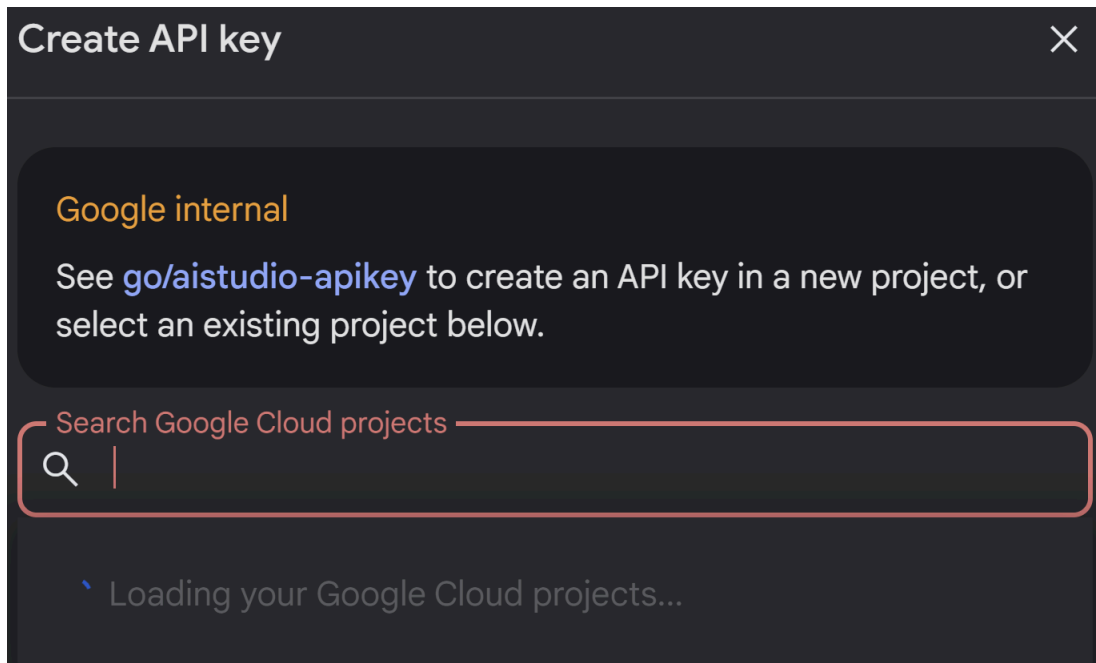
```
{
  "id": "62",
  "name": "image (61)",
  "description": "Pink toy of the Pink Panther with yellow eyes and black whiskers.",
  "quantity": "60",
  "price": "49.99",
  "image_url": "https://storage.googleapis.com/img_public_test/toys/image (61).png"
},
{
  "id": "59",
  "name": "image (58)",
  "description": "Pink toy of the Pink Panther with yellow eyes and black whiskers.",
  "quantity": "100",
  "price": "49.99",
  "image_url": "https://storage.googleapis.com/img_public_test/toys/image (58).png"
},
{
  "id": "60",
  "name": "image (59)",
  "description": "Pink toy of the Pink Panther with yellow eyes and black whiskers.",
  "quantity": "60",
  "price": "49.99",
  "image_url": "https://storage.googleapis.com/img_public_test/toys/image (59).png"
},
{
  "id": "61",
  "name": "image (60)",
  "description": "Pink toy of the Pink Panther with yellow eyes and black whiskers.",
  "quantity": "60",
  "price": "49.99",
  "image_url": "https://storage.googleapis.com/img_public_test/toys/image (60).png"
},
{
  "id": "63",
  "name": "image (62)",
  "description": "Pink toy of the Pink Panther with yellow eyes and black whiskers.",
  "quantity": "60",
  "price": "49.99",
  "image_url": "https://storage.googleapis.com/img_public_test/toys/image (62).png"
}
```

大功告成！在 AlloyDB 資料上使用 Embeddings 模型執行相似度向量搜尋，就是這麼簡單。

11. 建構網頁應用程式用戶端！

在本節中，我們將建構網頁應用程式，供使用者互動並根據文字、圖片尋找相符的玩具，甚至根據需求建立新玩具。由於應用程式已建構完成，請按照下列步驟將應用程式複製到 IDE，然後啟動並執行。

1. 由於我們會使用 Gemini 2.0 Flash 描述使用者上傳的圖片，以便尋找相符的玩具，因此需要取得這個應用程式的 API 金鑰。如要這麼做，請前往 <https://aistudio.google.com/apikey>，取得您要實作這個應用程式的有效 Google Cloud 專案 API 金鑰，並將金鑰儲存在某處：



2. 前往 Cloud Shell 終端機
3. 使用下列指令複製存放區：

```
git clone https://github.com/AbiramiSukumaran/toysearch  
  
cd toysearch
```

4. 存放區複製完成後，您應該就能從 [Cloud Shell 編輯器](#) 存取專案。
5. 您需要從複製的專案中刪除「get-toys-alloydb」和「toolbox-toys」資料夾，因為這 2 個資料夾是 Cloud Run Functions 程式碼，需要時可從存放區參照。
6. 前往 web 資料夾中的 GenerateToy.java，找出下列程式碼行並移除，因為允許成人內容可能需要特殊權限，部分試用帳單帳戶可能無法使用：

```
□ paramsMap.put("personGeneration", "allow_adult");  
□
```

7. 請務必先設定所有必要的環境變數，再建構及部署應用程式。前往 Cloud Shell 終端機並執行下列指令：

```
PROJECT_ID=$(gcloud config get-value project)

export PROJECT_ID=$PROJECT_ID

export GOOGLE_API_KEY=<YOUR API KEY that you saved>
```

8. 在本機建構及執行應用程式：

請確認您位於專案目錄中，然後執行下列指令：

```
mvn package

mvn spring-boot:run
```

9. 在 Cloud Run 上部署

```
gcloud run deploy --source .
```

步驟 12 · 約 0 分鐘

12. 瞭解生成式 AI 詳細資料

您不需要採取任何行動。僅供參考：

您現在已取得要部署的應用程式，請花點時間瞭解我們如何完成搜尋 (文字和圖片) 和生成作業。

1. 以使用者文字為基礎的向量搜尋：

我們在「使用向量搜尋應用程式網頁」一節中部署的 Cloud Run Functions，已解決這個問題。

2. 以圖片上傳為基礎的向量搜尋：

假設使用者想上傳一張熟悉玩具的圖片，並用這張圖片進行搜尋，而不是輸入文字。使用者可以上傳喜歡的玩具圖片，並取得相關功能。

我們運用 LangChain4j 叫用的 Google Gemini 2.0 Flash 模型分析圖片，並擷取相關脈絡，例如玩具的顏色、材質、類型和適用年齡層。

我們只用了 5 個步驟，就透過開放原始碼架構，以大型語言模型叫用功能，將使用者輸入的多模態資料與相符結果配對。瞭解如何：

```

package cloudcode.helloworld.web;

import dev.langchain4j.model.chat.ChatLanguageModel;
import dev.langchain4j.model.googleai.GoogleAiGeminiChatModel;
import dev.langchain4j.data.message.UserMessage;
import dev.langchain4j.data.message.AiMessage;
import dev.langchain4j.model.output.Response;
import dev.langchain4j.data.message.ImageContent;
import dev.langchain4j.data.message.TextContent;
import java.util.Base64;
import java.util.Optional;

public class GeminiCall {
    public String imageToBase64String(byte[] imageBytes) {
        String base64Img = Base64.getEncoder().encodeToString(imageBytes);
        return base64Img;
    }

    public String callGemini(String base64ImgWithPrefix) throws Exception {
        String searchText = "";

        // 1. Remove the prefix
        String base64Img = base64ImgWithPrefix.replace("data:image/jpeg;base64,", "");

        // 2. Decode base64 to bytes
        byte[] imageBytes = Base64.getDecoder().decode(base64Img);
        String image = imageToBase64String(imageBytes);

        // 3. Get API key from environment variable
        String apiKey = Optional.ofNullable(System.getenv("GOOGLE_API_KEY"))
            .orElseThrow(() -> new IllegalArgumentException("GOOGLE_API_KEY environment
variable not set"));

        // 4. Invoke Gemini 2.0
        ChatLanguageModel gemini = GoogleAiGeminiChatModel.builder()
            .apiKey(apiKey)
            .modelName("gemini-2.0-flash-001")
            .build();

        Response<AiMessage> response = gemini.generate(
            UserMessage.from(
                ImageContent.from(image, "image/jpeg"),
                TextContent.from(
                    "The picture has a toy in it. Describe the toy in the image in one line. Do
not add any prefix or title to your description. Just describe that toy that you see in the
image in one line, do not describe the surroundings and other objects around the toy in the
image. If you do not see any toy in the image, send response stating that no toy is found in
the input image.")));

        // 5. Get the text from the response and send it back to the controller
        searchText = response.content().text().trim();
        System.out.println("searchText inside Geminicall: " + searchText);
    }
}

```

```
    return searchText;
  }
}
```

3. 瞭解我們如何使用 Imagen 3，根據使用者要求，透過生成式 AI 建構客製化玩具。

Imagen 3 接著會生成自訂設計玩具的圖片，讓使用者清楚看到自己的創作。我們只用了 5 個步驟就完成這項作業：

```

// Generate an image using a text prompt using an Imagen model
public String generateImage(String projectId, String location, String prompt)
    throws ApiException, IOException {
    final String endpoint = String.format("%s-aiplatform.googleapis.com:443", location);
    PredictionServiceSettings predictionServiceSettings =
        PredictionServiceSettings.newBuilder().setEndpoint(endpoint).build();

    // 1. Set up the context and prompt
    String context = "Generate a photo-realistic image of a toy described in the following
input text from the user. Make sure you adhere to all the little details and requirements
mentioned in the prompt. Ensure that the user is only describing a toy. If it is anything
unrelated to a toy, politely decline the request stating that the request is inappropriate
for the current context. ";
    prompt = context + prompt;

    // 2. Initialize a client that will be used to send requests. This client only needs to
be created
    // once, and can be reused for multiple requests.
    try (PredictionServiceClient predictionServiceClient =
        PredictionServiceClient.create(predictionServiceSettings)) {

    // 3. Invoke Imagen 3
    final EndpointName endpointName =
        EndpointName.ofProjectLocationPublisherModelName(
            projectId, location, "google", "imagen-3.0-generate-001");
    //"imagegeneration@006"; imagen-3.0-generate-001
    Map<String, Object> instancesMap = new HashMap<>();
    instancesMap.put("prompt", prompt);
    Value instances = mapToValue(instancesMap);
    Map<String, Object> paramsMap = new HashMap<>();
    paramsMap.put("sampleCount", 1);
    paramsMap.put("aspectRatio", "1:1");
    paramsMap.put("safetyFilterLevel", "block_few");
    paramsMap.put("personGeneration", "allow_adult");
    paramsMap.put("guidanceScale", 21);
    paramsMap.put("imagenControlScale", 0.95); //Setting imagenControlScale
    Value parameters = mapToValue(paramsMap);

    // 4. Get prediction response image
    PredictResponse predictResponse =
        predictionServiceClient.predict(
            endpointName, Collections.singletonList(instances), parameters);

    // 5. Return the Base64 Encoded String to the controller
    for (Value prediction : predictResponse.getPredictionsList()) {
        Map<String, Value> fieldsMap = prediction.getStructValue().getFieldsMap();
        if (fieldsMap.containsKey("bytesBase64Encoded")) {
            bytesBase64EncodedOutput = fieldsMap.get("bytesBase64Encoded").getStringValue();
        }
    }
    return bytesBase64EncodedOutput.toString();
}

```

```
}  
}
```

價格預測

在上述章節中，我們討論了 Imagen 如何生成使用者想自行設計的玩具圖片。應用程式必須為該玩具設定價格，使用者才能購買。我們採用直覺式邏輯，為客製化訂製玩具定義價格。系統會根據使用者設計的玩具，找出描述最接近的前 5 個玩具，並計算平均價格。

生成玩具的價格預測是這個應用程式的重要環節，我們採用代理式方法生成這項資訊。隆重推出「資料庫專用的生成式 AI 工具箱」。

步驟 13 · 約 0 分鐘

13. 資料庫專用的生成式 AI 工具箱

「資料庫專用的生成式 AI 工具箱」是 Google 的開放原始碼伺服器，可簡化與資料庫互動的生成式 AI 工具建構程序。這項服務可處理連線集區和驗證等複雜作業，讓您更輕鬆、快速且安全地開發工具。這項工具可協助您建構生成式 AI 工具，讓 AI 代理能存取資料庫中的資料。

如要設定這項功能，讓工具準備就緒並讓應用程式具備代理功能，請按照下列步驟操作：[工具箱程式碼研究室連結](#)

現在，應用程式可以使用這個已部署的 Cloud Run 函式端點，填入價格，以及為客製化訂製玩具圖片生成的 Imagen 結果。

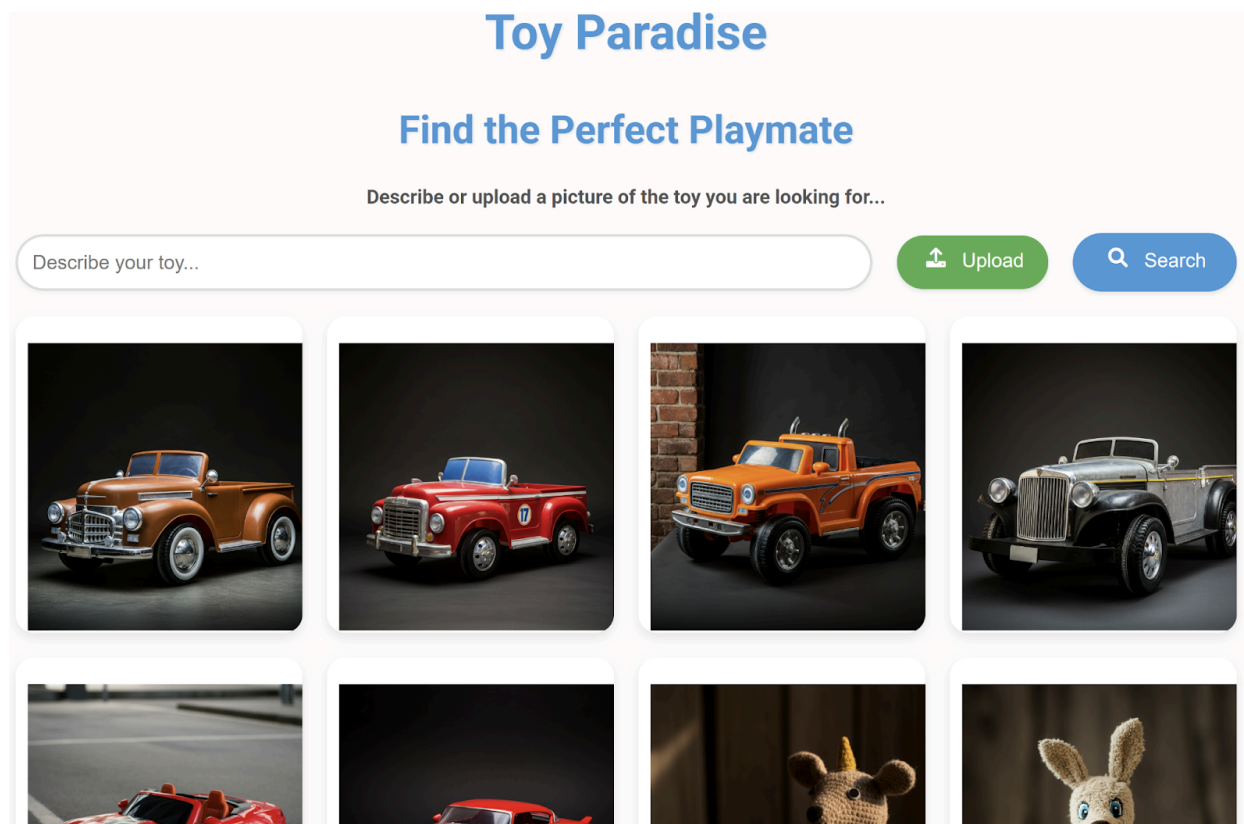
步驟 14 · 約 0 分鐘

14. 測試網頁應用程式

應用程式的所有元件都已建構及部署完成，現在可以透過雲端提供服務。測試應用程式是否能處理所有情境。以下是影片連結，可讓您瞭解實際情況：

<https://www.youtube.com/shorts/ZMqUAWsghYQ>

到達網頁如下所示：



步驟 15 · 約 0 分鐘

15. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
-

16. 恭喜

恭喜！您已成功使用 AlloyDB、pgvector、Imagen 和 Gemini 2.0 執行 Toystore 脈絡搜尋和生成作業，同時運用開放原始碼程式庫建構穩固的整合功能。結合 [AlloyDB](#)、[Vertex AI](#) 和 [Vector Search](#) 的功能，我們在提供情境和向量搜尋功能方面取得重大進展，讓這類搜尋功能更易於使用、效率更高，且真正以意義為導向。
